Code walkthrough

data_memory.v

Data Memory

What it does	Stores data for LOAD and STORE (16 words x 4 bits)
Where it works	EX stage
Size	41 lines, explained in 6 small parts
Modules in this file	data_memory
Calls (uses)	nothing (no other modules)
Called by	cpu_top.v - instance DMEM tb_cpu.v - reads and clears CPU.DMEM.mem

The data memory is a small RAM. LOAD reads from it and STORE writes to it. It is separate from the instruction memory (Harvard organisation), so a store can never damage the program.

Map of the file

Part	Lines	What this part is
1	1-16	Header comment
2	17-23	Ports
3	24-25	The storage
4	27-31	Start-up values
5	33-37	Writing
6	39-41	Reading

> How to read this document: each part below shows a few lines of the real file, with their line numbers, followed by a plain-language explanation. Read the parts in order to understand the whole file without scrolling through it in one go.

Part 1 · Header comment (lines 1-16)

data_memory.v lines 1-16

```
 1  // ======================================================================
 2  // FILE: data_memory.v
 3  // Data RAM: 16 words x 4 bits. Used by LOAD and STORE.
 4  //
 5  // Interface (as in the project plan):
 6  //   clk, address, write_data, mem_we, read_data
 7  //
 8  // - Read  : any time (no clock needed)          -> LOAD
 9  // - Write : on the clock edge when mem_we = 1    -> STORE
10  // It is separate from the instruction memory (Harvard organisation).
11  //
12  // CONNECTIONS
13  //   Calls       : nothing (no other modules)
14  //   Called by   : cpu_top.v - instance DMEM
15  //                 tb_cpu.v - reads and clears CPU.DMEM.mem
16  // ======================================================================
```

- Size, interface and the rule: read any time, write only on a clock edge.
- The **CONNECTIONS** lines at the end of the header list which other files this file uses (Calls) and which files use it (Called by).

Part 2 · Ports (lines 17-23)

data_memory.v lines 17-23

```
17  module data_memory (
18      input        clk,
19      input  [3:0] address,         // which word
20      input  [3:0] write_data,      // data to write (STORE)
21      input        mem_we,          // write enable: 1 = write
22      output [3:0] read_data        // data read (LOAD)
23  );
```

- `address`: which word (comes from the ALU result).
- `write_data`: value to store (the value of Rd).
- `mem_we`: write enable, 1 only for STORE.
- `read_data`: the value at the address (used by LOAD).

Part 3 · The storage (lines 24-25)

data_memory.v lines 24-25

```
24      reg [3:0] mem [0:15];         // 16 words, 4 bits each
25      integer i;
```

16 words of 4 bits each, plus a loop counter.

Part 4 · Start-up values (lines 27-31)

data_memory.v lines 27-31

```
27      // start with all words = 0 (no unknown X values)
28      initial begin
29          for (i = 0; i < 16; i = i + 1)
30              mem[i] = 4'b0000;
31      end
```

All 16 words are set to 0 at time 0 so that a LOAD from an unused address gives 0, not an unknown value.

Part 5 · Writing (lines 33-37)

data_memory.v lines 33-37

```
33      // write on the clock edge
34      always @(posedge clk) begin
35          if (mem_we)
36              mem[address] <= write_data;
37      end
```

At each rising clock edge, if `mem_we` is 1, the word at `address` is replaced by `write_data`. If mem_we is 0 nothing changes.

Part 6 · Reading (lines 39-41)

data_memory.v lines 39-41

```
39      // read any time
40      assign read_data = mem[address];
41  endmodule
```

`assign read_data = mem[address];` The stored value appears at once, so LOAD can read and write the register in the same EX cycle.

> **In short:** Read any time; write only when mem_we = 1 at a clock edge. **Example:** STORE R1, 0 with R1 = 1000: address = 0000, write_data = 1000, mem_we = 1, so mem[0] becomes 1000 at the clock edge. Later LOAD R0, 0 sets address = 0000 and read_data shows 1000 straight away.